

SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-004: Student Input Validation and Strong Password Enforcement

Field	Details
Change Request ID	CR-004
Change Request Title	Student Input Validation and Strong Password Enforcement
Student Name	[Student Name]
Student ID	[Student ID]
Implementation Status	Completed
Date	8 October 2026

1. Change Request Summary

1.1 Business Requirement

Administrators previously could submit student information without sufficient validation. This could result in invalid names, incorrect phone numbers, invalid email addresses, and weak student passwords being stored in the system.

The change was required to improve student data quality and account security by introducing validation for student Add and Edit operations.

1.2 Existing Behaviour

Before implementation:

- The administrator could add and edit student records.
- Basic required-field validation was available.
- First and last names did not have a minimum character requirement.
- Phone number validation was not strict enough.
- Email validation was mainly dependent on the HTML input type.
- Strong password requirements were not enforced on the backend.
- Invalid data could potentially be submitted directly through API requests.
- Passwords were already protected using bcrypt hashing.

1.3 Requested Change

The system was required to validate student information during Add and Edit operations.

The validation must:

- Require a valid registration number.
- Require first name with at least 3 characters.
- Require last name with at least 3 characters.
- Allow only valid characters for student names.
- Validate email format.
- Require exactly 10 numeric digits for the phone number.
- Require a password when creating a new student.
- Require passwords to contain at least 8 characters.
- Require uppercase and lowercase letters in the password.

- Require at least one number.
- Require at least one special character.
- Perform validation on both frontend and backend.
- Prevent invalid API requests from bypassing validation.
- Preserve existing Add, Edit, and Delete functionality.
- Continue using bcrypt password hashing.

2. Implementation

2.1 Implementation Summary

The CR-004 requirements were implemented at two validation levels.

Client-side validation was added to StudentFormModal.jsx to provide immediate feedback to administrators and prevent invalid form submission.

Server-side validation was added to studentController.js to prevent validation bypass through direct API requests.

The existing studentModel.js was preserved because it is responsible for database operations. No database schema changes were required.

The existing bcrypt password hashing mechanism was also preserved.

2.2 Affected Files

File	Change	Purpose
frontend/src/components/StudentFormModal.jsx	Modified	Added client-side validation for student fields and strong password validation
backend/controllers/studentController.js	Modified	Added server-side validation for student

		fields and password requirements
backend/models/studentModel.js	Unchanged	Existing database operations remain unchanged
Database	Unchanged	No schema or table changes required

2.3 Technical Changes

Student Name Validation

First and last names are validated to ensure:

- The value is not empty.
- The name contains at least 3 characters.
- Invalid characters are rejected.

This prevents short or invalid names from being submitted.

Email Validation

The email field is validated using an email format check.

Invalid email addresses are rejected before the request is processed.

Phone Number Validation

The phone number is validated to ensure:

- Exactly 10 digits are provided.
- Only numeric characters are accepted.

This prevents invalid phone numbers from being stored.

Registration Number Validation

Registration number is required and cannot be submitted as an empty value.

Strong Password Validation

For new student accounts, the password must:

- Contain at least 8 characters.
- Contain at least one uppercase letter.
- Contain at least one lowercase letter.
- Contain at least one number.
- Contain at least one special character.

Example of a valid strong password:

Student@123

Client-Side Validation

StudentFormModal.jsx validates the data before sending the API request.

If invalid data is entered, a clear validation message is displayed and the API request is not sent.

Server-Side Validation

studentController.js performs the same critical validation before database operations.

This prevents users from bypassing frontend validation by directly sending requests to the API.

Password Security

The existing bcrypt password hashing mechanism remains unchanged.

Passwords are not stored as plain text.

2.4 Implementation Logic

1. Administrator opens the Add Student or Edit Student form.
2. Administrator enters student information.
3. Client-side validation checks the entered values.
4. If validation fails, an appropriate error message is displayed.
5. If validation succeeds, the API request is sent.
6. The backend receives the request.
7. Server-side validation checks the submitted values again.
8. Invalid requests are rejected with an appropriate HTTP error response.

9. Valid student information is processed normally.
10. During student creation, the password is hashed using bcrypt.
11. The student account and profile are created.
12. Existing Edit and Delete functionality continues to use the correct student_id.

2.5 Error and Data Handling

The implementation handles invalid input before database operations.

Examples include:

- Empty required fields.
- Names shorter than 3 characters.
- Invalid name characters.
- Invalid email addresses.
- Phone numbers containing non-numeric characters.
- Phone numbers with fewer or more than 10 digits.
- Empty passwords during student creation.
- Passwords that do not satisfy the strong password policy.

Clear validation messages are returned to the administrator.

Server-side validation ensures that invalid data cannot be stored simply by bypassing the frontend.

Area	Impact	Details
Frontend	Low	Dashboard search/filtering logic and UI added.
Backend	None	Existing API remains unchanged.
Database	None	No schema or data changes required.
API	None	Existing GET /api/students/ remains in use.
Authentication / Authorization	None	Existing authentication and authorization remain unchanged.

Security	Low	Search does not expose additional sensitive information.
User Interface / Styling	Low	Search input, result count, Clear Search, and no-result message added.
Configuration	None	No configuration changes required.
External Services	None	No external services used.
Performance	Low	Filtering is performed locally on already-loaded records.
Testing	Low	Functional, negative, CRUD, and regression testing performed.
Documentation	Low	Implementation and testing documented in this report.

3. Testing & Verification

3.1 Testing Approach

The implementation was verified through frontend and backend testing.

Testing Types:

- Functional testing
- Positive testing
- Negative testing
- Boundary testing
- Validation testing
- API testing
- Frontend testing
- Backend testing
- Security testing
- Error-handling testing
- CRUD verification
- Regression testing

Environment: Development

Browser: [Chrome / Edge / Firefox]

Test Role: Administrator

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Submit empty required fields	Validation error displayed	Validation error displayed	Pass
TC-02	Enter first name with fewer than 3 characters	First name rejected	First name rejected	Pass
TC-03	Enter valid first name with 3 or more characters	First name accepted	First name accepted	Pass
TC-04	Enter last name with fewer than 3 characters	Last name rejected	Last name rejected	Pass
TC-05	Enter invalid characters in name	Invalid name rejected	Invalid name rejected	Pass
TC-06	Enter invalid email address	Email rejected	Email rejected	Pass
TC-07	Enter valid email address	Email accepted	Email accepted	Pass
TC-08	Enter phone number with fewer than 10 digits	Phone number rejected	Phone number rejected	Pass
TC-09	Enter phone number with exactly 10 digits	Phone number accepted	Phone number accepted	Pass
TC-10	Enter phone number with more than 10 digits	Phone number rejected	Phone number rejected	Pass
TC-11	Enter non-numeric phone number	Phone number rejected	Phone number rejected	Pass

TC-12	Create student without password	Password validation error displayed	Password validation error displayed	Pass
TC-13	Enter password shorter than 8 characters	Password rejected	Password rejected	Pass
TC-14	Enter password without uppercase letter	Password rejected	Password rejected	Pass
TC-15	Enter password without lowercase letter	Password rejected	Password rejected	Pass
TC-16	Enter password without number	Password rejected	Password rejected	Pass
TC-17	Enter password without special character	Password rejected	Password rejected	Pass
TC-18	Enter valid strong password	Password accepted	Password accepted	Pass
TC-19	Create student using valid information	Student created successfully	Student created successfully	Pass
TC-20	Edit student using valid information	Student updated successfully	Student updated successfully	Pass
TC-21	Edit student using invalid information	Validation error displayed	Validation error displayed	Pass
TC-22	Send invalid data directly to API	Server rejects invalid request	Server rejects invalid request	Pass
TC-23	Delete existing student	Correct student deleted	Correct student deleted	Pass
TC-24	Verify password storage	Password is hashed using bcrypt	Password stored securely	Pass

3.3 Test Summary

Test Result	Count
Total Test Cases	24
Passed	24
Failed	0
Blocked	0
Not Tested	0
Pass Rate	100%

Overall Test Result: Pass

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification	Status
AC-001	Required student fields are validated	Frontend and backend validation test	Met
AC-002	First and last names require at least 3 characters	Name validation test	Met
AC-003	Invalid name characters are rejected	Negative validation test	Met
AC-004	Email format is validated	Email validation test	Met
AC-005	Phone number contains exactly 10 numeric digits	Phone validation test	Met

AC-006	New student passwords require at least 8 characters	Password validation test	Met
AC-007	Password contains uppercase, lowercase, number and special character	Strong password test	Met
AC-008	Backend prevents validation bypass	Direct API validation test	Met
AC-009	Password is not stored as plain text	Password hashing verification	Met
AC-010	Existing Add, Edit and Delete functionality continues to work	CRUD regression testing	Met
AC-011	Existing authentication and authorization remain unchanged	Administrator access testing	Met

5. Evidence

A maximum of relevant screenshots should be attached.

Evidence 01 – Student Form Validation

Related: AC-001, AC-002, AC-003, AC-004, AC-005

Figure 1: Administrator student form displaying validation errors for invalid student information.

Add New Student



Create a new student account.

Phone number must contain exactly 10 digits.

Registration Number

REG004

First Name

Manoja

Last Name

Manamatharasa

Email

Mano@gmail.com

Phone Number

2

Password

.....

Cancel

Create Student

Add New Student

Create a new student account.



Phone number must contain exactly 10 digits.

Registration Number

REG004

First Name

Last Name

Manamatharasa

Email

Mano@gmail.com

Phone Number

1234567890

Password

.....

Cancel

Create Student

! Please fill out this field.

Add New Student

Create a new student account.



Phone number must contain exactly 10 digits.

Registration Number

REG004

First Name

M

Last Name

Manamatharasa



Please lengthen this text to 3 characters or more (you are currently using 1 character).

Phone Number

1234567890

Password

.....

Cancel

Create Student

Students

Manage student accounts and profiles.

+ Add Student

Search by registration number, name, or email

3 students

REGISTRATION NUMBER	NAME	EMAIL	ACTIONS
REG003	Manoja	Manoja@gmail.com	Delete
REG002			Delete
REG001			Delete

Add New Student

Create a new student account.

Last name must contain only letters.

Registration Number	First Name
REG004	Manoja
Last Name	Email
--09	Mano@gmail.com
Phone Number	Password
1234567890	*****

Cancel Create Student

Students

Manage student accounts and profiles.

+ Add Student

Search by registration number, name, or email

3 students

REGISTRATION NUMBER	NAME	EMAIL	ACTIONS
REG003	Manoja	Manoja@gmail.com	Delete
REG002			Delete
REG001			Delete

Add New Student

Create a new student account.

Please include an '@' in the email address. 'Manojagmail.com' is missing an '@'.

Registration Number	First Name
REG003	Manoja
Last Name	Email
Rani	Manojagmail.com
Phone Number	Password
1234567890	*****

Cancel Create Student

Evidence 02 – Strong Password Validation

Related: AC-006, AC-007

Figure 2: Student creation form displaying strong password validation requirements and validation feedback.

The screenshot displays a web application for managing students. The main heading is "Students" with a subtitle "Manage student accounts and profiles." and a "+ Add Student" button. A search bar is present with the placeholder "Search by registration number, name, or email...". Below the search bar, a list of students is shown with columns for "REGISTRATION NUMBER" and "ACTIONS". The list contains three entries: REG003, REG002, and REG001, each with a "Delete" button.

An "Add New Student" modal form is open, titled "Add New Student" with a close button (X). The form includes the instruction "Create a new student account." and the following fields:

- Registration Number: REG003
- First Name: Manoja
- Last Name: Rani
- Email: Manoja@gmail.com
- Phone Number: 1234567890
- Password: ...

A validation error message is displayed below the Password field:

! Please lengthen this text to 8 characters or more (you are currently using 3 characters).

Students

Manage student accounts and profiles.

+ Add Student

Search by registration

3 students

REGISTRATION N

REG003

REG002

REG001

IONS

Delete

Delete

Delete

Add New Student

×

Create a new student account.

Password must contain at least one uppercase letter.

Registration Number

REG003

First Name

Manoja

Last Name

Rani

Email

Manoja@gmail.com

Phone Number

1234567890

Password

.....

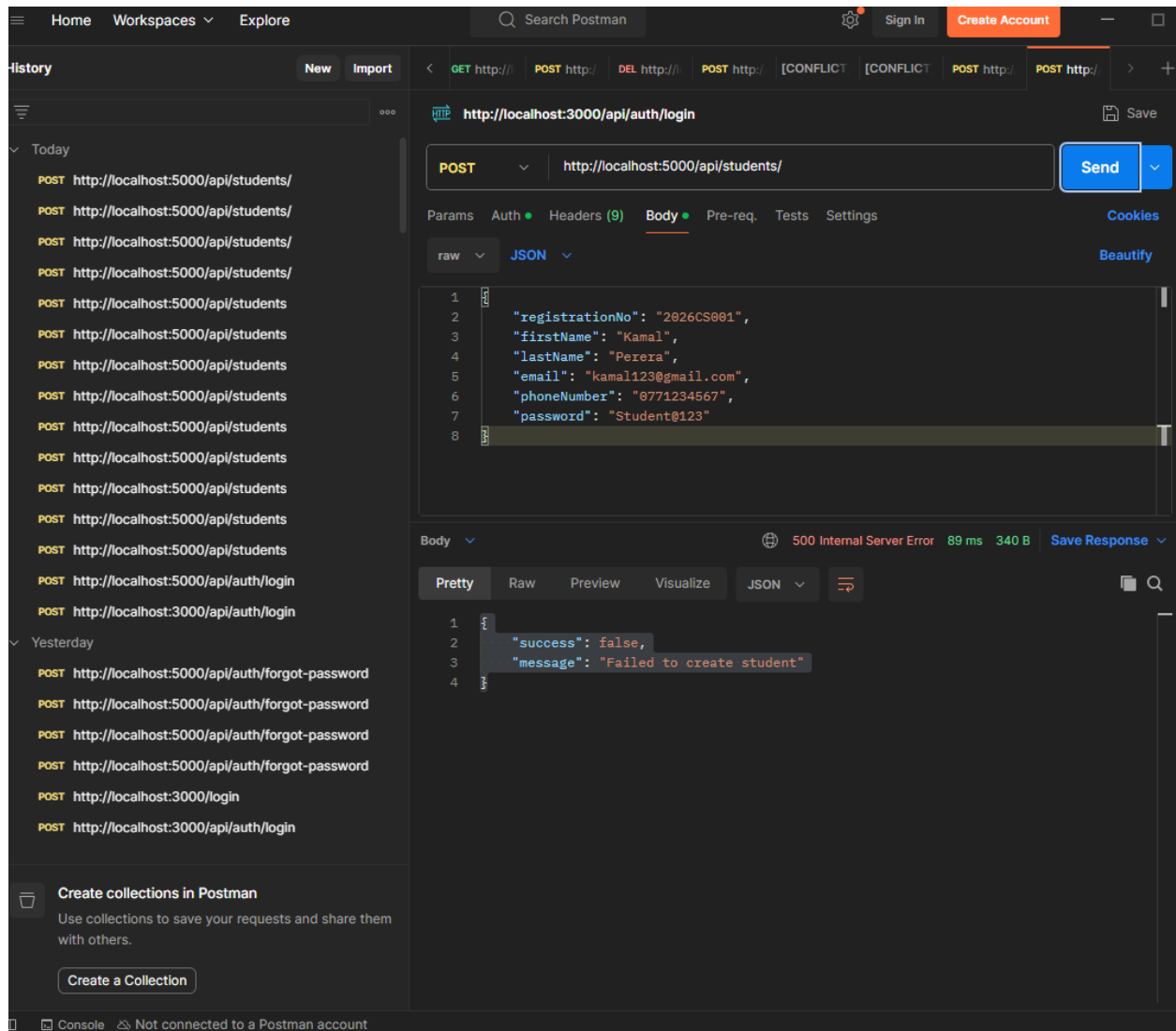
Cancel

Create Student

Evidence 03 – Backend Validation

Related: AC-008

Figure 3: API request containing invalid student information rejected by server-side validation.



Evidence 04 – Successful Student Creation

Related: AC-009, AC-010

Figure 4: Successfully created student with password securely hashed in the database and existing CRUD functionality working correctly.

Student Management System

Administrator Dashboard

admin
ADMIN

Logout

Students

Manage student accounts and profiles.

+ Add Student

3 students

REGISTRATION NO.	NAME	EMAIL	PHONE	ACTIONS
REG003	Maha Ranii	Maha@gmail.com	0778985832	EditDelete
REG002	Sarah Fernando	sarah@example.com	0723456789	EditDelete
REG001	John Perera	john@example.com	0712345678	EditDelete

6. Final Result

Item	Result
Requirements	Met
Implementation	Completed
Testing	Passed
Acceptance Criteria	All Met
Existing CRUD Verified	Verified
Password Security	Verified
Final Status	Accepted

6.1 Final Summary

CR-004 was implemented as a student input validation and strong password enforcement enhancement.

Client-side validation was added to StudentFormModal.jsx to provide immediate validation feedback and prevent invalid form submission.

Server-side validation was added to studentController.js to prevent validation bypass through direct API requests.

The implemented validation covers registration number, first name, last name, email address, phone number, and student password requirements.

First and last names require at least 3 characters, phone numbers require exactly 10 numeric digits, and new student passwords require at least 8 characters containing uppercase letters, lowercase letters, numbers, and special characters.

The existing bcrypt password hashing mechanism was preserved to ensure passwords are not stored as plain text.

The existing studentModel.js, database structure, API endpoints, authentication, authorization, and CRUD functionality were preserved.

The implementation was tested using positive, negative, boundary, API, security, and regression test cases. All defined test cases passed and all acceptance criteria were met.

6.2 Issues / Limitations

No significant issues identified.

Student Name: S.Hindusha

Student ID: JFICT24_26

Date: 8 October 2026

Signature: _____

~ END OF REPORT ~